Smart E-Commerce Recommendation and Customer Analytics System

A practical database systems and analytics assignment

Assignment goal: Design an end-to-end system that uses MySQL to organize and analyze e commerce data, then uses Python to produce customer segments and explainable product recommendation

Background

An e-commerce company stores customer, product, order, payment, and browsing data, but the data is not yet being used to support business decisions. Management wants a system that can explain customer purchasing behavior, identify valuable or inactive customers, measure product performance, and recommend relevant products.

Your task is to design and implement an end-to-end analytics system using MySQL and Python.

1. Main Problem

Build a system that answers the following question:

> **What does each customer buy, how valuable are they, what are they likely to buy next, and which customers or products need attention?**

The finished project must show how data moves from a relational database into Python analysis and then returns as useful results and reports.

2. Learning Outcomes

After completing this project, you should be able to:

1. design a normalized relational database;
2. apply keys, constraints, relationships, and suitable data types;
3. write basic and advanced analytical SQL;

4. connect Python to MySQL safely;
5. analyze customer behavior with Pandas;
6. perform RFM customer segmentation;
7. build a simple, explainable recommendation system;
8. save analytical results and present them clearly.

3. Required Database Design

Create a MySQL database containing at least these entities:

- customers;
- categories;
- products;
- orders;
- order items;
- payments.

Reviews and product-view events may also be included when they support the analysis. The main relationship path should be:

```
Customer -> Order -> Order Item -> Product -> Category
```

The schema must use appropriate:

- primary and foreign keys;
- `NOT NULL`, `UNIQUE`, `CHECK`, and `DEFAULT` constraints;
- numeric, text, date, and status data types;
- indexes for real filtering and joining needs;
- transactions where multiple database changes must succeed together.

Use realistic demonstration data so every major part of the system can be tested.

4. Customer Analytics Requirements

Use SQL to calculate:

- total spending for each customer;
- number of completed orders;
- average order value;
- most recent purchase date;
- most frequently purchased category.

The analysis must identify:

- high-value customers with strong spending and purchase frequency;
- customers who have not purchased within a chosen period;
- customers at risk of becoming inactive;
- one-time customers;
- repeat customers;
- prospects who have not purchased yet.

5. Product Analytics Requirements

Use SQL to determine:

- best-selling products;
- highest-revenue and most profitable products;
- best-performing categories;
- products frequently purchased together;
- products with declining monthly sales;
- products with many views but low purchase conversion, when view data is available.

6. Required Business Questions

The completed system should answer all of the following.

Customer questions

1. What and how often does each customer buy?
2. Who are the top customers by spending?
3. Who is a prospect, one-time buyer, or repeat customer?
4. Which customers are inactive?
5. Which customers are at risk of becoming inactive?
6. What is the average observed customer lifetime value?
7. What is each customer's favorite product category?
8. Which customers have above-average spending and purchase frequency?

Product questions

9. Which products sell the most and generate the most profit?
10. Which categories generate the most revenue and profit?
11. Which products are frequently purchased together?
12. Which products are losing sales compared with the previous month?
13. Which products have many views but poor purchase conversion?

Recommendation and segmentation questions

14. What should customer `C101` buy next, and why?
15. How many customers may be interested in each recommended product?
16. Which customers belong to each RFM segment?

7. Advanced SQL Requirements

Use the following techniques where they genuinely help solve the problem:

- `JOIN` and `LEFT JOIN`;
- product-pair `SELF JOIN`;
- `GROUP BY` and `HAVING`;
- subqueries;
- common table expressions;
- `CASE` expressions;
- window functions;
- reusable views;
- indexes;
- transactions.

Do not include a technique only to tick a box. Its use should be connected to a business requirement.

8. Python Analytics Requirements

Python must connect directly to MySQL and perform the following steps:

1. retrieve customer, order, and purchase data;
2. clean and organize the data with Pandas;
3. calculate RFM values and scores;
4. classify customers as `VIP`, `Regular`, `Occasional`, or `Inactive`;
5. create personalized product recommendations;
6. exclude products the customer already owns;
7. include a readable reason for every recommendation;
8. save results to MySQL and export them as CSV reports.

Use a transparent recommendation method such as customer-based collaborative filtering or item-based collaborative filtering. Deep learning is not required.

9. Required Deliverables

Submit the following:

1. database schema and setup scripts;
2. realistic seed data;
3. SQL views and business-analysis queries;
4. Python source code for database access, RFM, and recommendations;
5. result tables and generated CSV reports;
6. unit tests for the main Python logic;
7. a README with setup and demonstration instructions;
8. an ER diagram;
9. a short demonstration showing the main results.

Credentials, local environment files, and virtual environments must not be submitted.